Hospital Emergency Queue - Bubble Sort Using Arrays

1. Real-Time Scenario

CityCare Multi-Specialty Hospital receives hundreds of emergency patients every day. Each patient is assigned an Emergency Level from 1 to 5. Level 1 - Critical (Immediate Treatment) Level 2 - Serious Level 3 - Moderate Level 4 - Minor Level 5 - Stable Due to a system issue, patient records are stored in random order. Arrange the patients so doctors always treat the most critical patients first. Use only Pure Arrays and Bubble Sort.

2. Input

```
Patient ID    Name      Level
P101          Rahul       3
P102          Priya       1
P103          Amit        5
P104          Sneha       2
P105          Arjun       4
```

3. Visual Representation

```
Current Queue

■ Arrival
      |
      V

+------------------------+
| Rahul      Level 3     |
| Priya      Level 1     |
| Amit       Level 5     |
| Sneha      Level 2     |
| Arjun      Level 4     |
+------------------------+

Goal

+------------------------+
| Priya      Level 1     |
| Sneha      Level 2     |
| Rahul      Level 3     |
| Arjun      Level 4     |
| Amit       Level 5     |
+------------------------+
```

4. Bubble Sort Dry Run

```
Initial

3   1   5   2   4

Pass 1

3 > 1  -> Swap

1   3   5   2   4

5 > 2  -> Swap

1   3   2   5   4

5 > 4  -> Swap

1   3   2   4   5
Pass 2
```

```
3 > 2 -> Swap

1 2 3 4 5

Queue Sorted
```

5. Pseudo Algorithm

```
START

Read PatientID[]
Read PatientName[]
Read EmergencyLevel[]

FOR each pass

   Compare adjacent levels

   IF left > right

      Swap EmergencyLevel
      Swap PatientID
      Swap PatientName

   END IF

Repeat until sorted

Display treatment queue

STOP
```

6. Key Idea

When two emergency levels are swapped, the corresponding Patient ID and Patient Name must also be swapped because they belong to the same patient record (parallel arrays).

7. Interview Tips

• Why use Bubble Sort? • Why are three arrays swapped together? • What happens if two patients have the same emergency level? • Time Complexity: $O(n^2)$ • Space Complexity: $O(1)$